

Recursion

It is a method in which a fⁿ calls itself usually with smaller inputs.

$f(n) \rightarrow f(n-1) \rightarrow f(n-2) \rightarrow f(n-3) \dots$

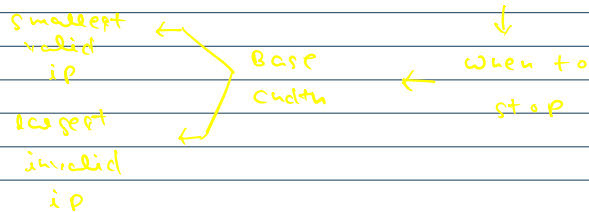
$A() \rightarrow B() \{ A \text{ calls } B \} \{ \text{Function calling} \}$

$A() \rightarrow A() \{ A \text{ calls } A \} \{ \text{Function calling itself} \}$

Where do we need recursion?

Choices + Decisions

$f(n) \rightarrow f(n-1) \rightarrow f(n-2) \rightarrow f(n-3) \dots ?$



"Recursion works like magic"

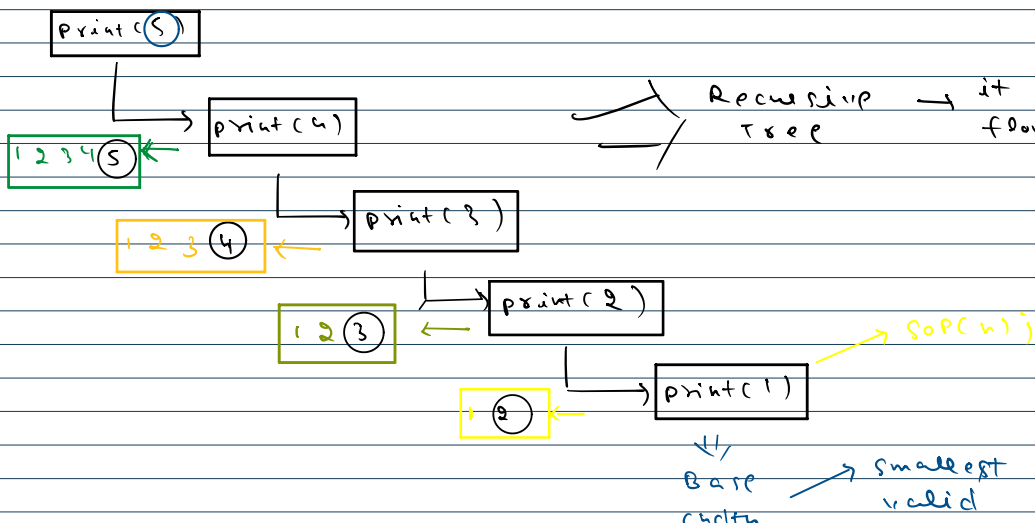
$f(n) \rightarrow f(n-1)$

↓
Trust that $f(n-1)$ will always return a valid op.

Print numbers from 1 to n using Recursion

Input $\Rightarrow n = 5$

Output $\Rightarrow$
1
2
3
4
5

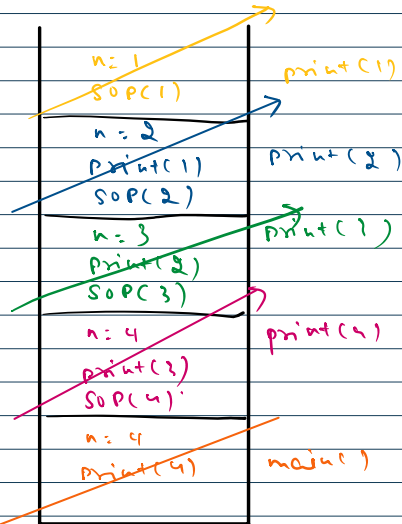


Recursion tree $\rightarrow$ it visualizes the code flow of a recursive code.

Base
case
↗ smallest
valid
i/p

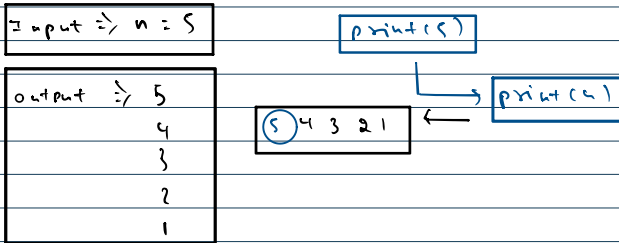
```
public class PrintIncreasing {
    public static void main(String args[]) {
        int n = 4;
        printIncr(n);
    }

    2 usages
    private static void printIncr(int n) {
        if(n == 1) {
            System.out.println(n);
            return;
        }
        printIncr(n-1);
        System.out.println(n);
    }
}
```



1
2
3
4
console

Print numbers from n to 1 using Recursion

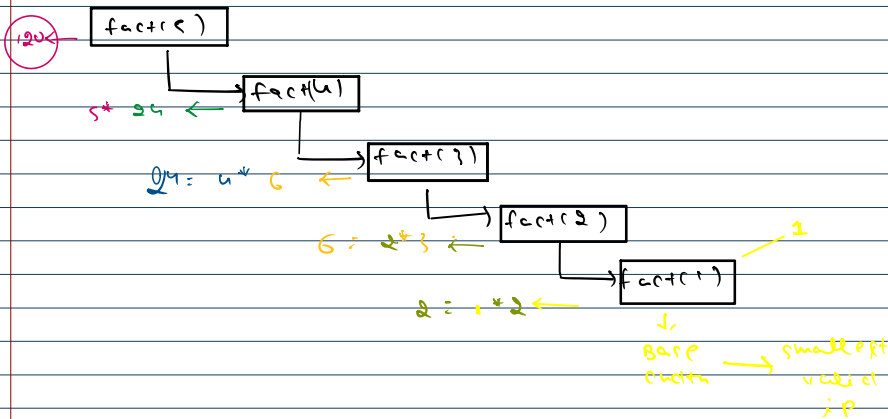


Factorial of a number

$$5! = 5 \times 4!$$

$$n! = n \times (n-1)!$$

$$\text{fact}(n) = n * \text{fact}(n-1)$$



First Occurrence of a number in an array

arr → { 1, 2, 3, 4, 3, 4, 5, 6, 7, 8, 9 }
item → 4

o/p → 3

recursion

o/p → 3

check if an array is sorted or not

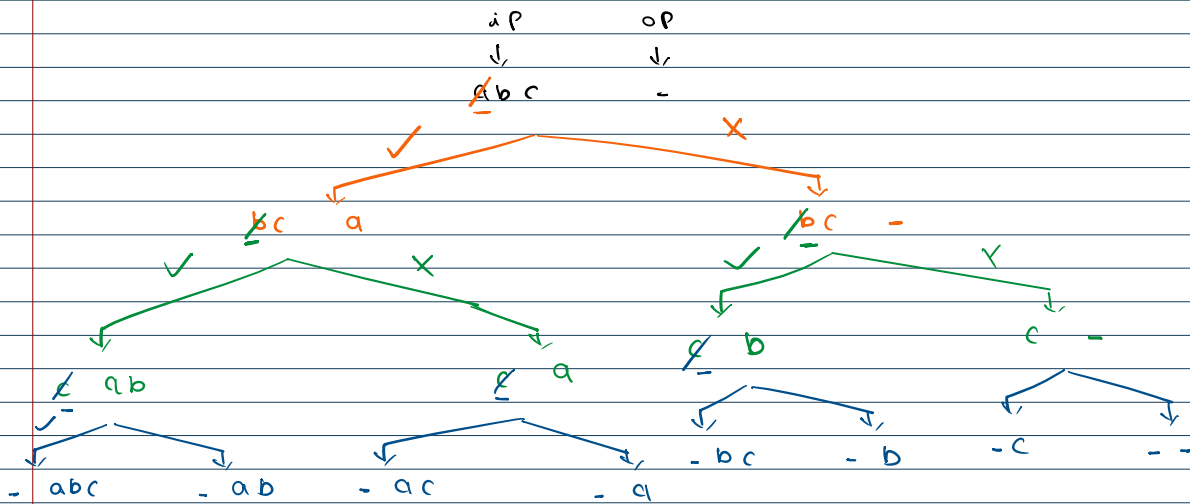
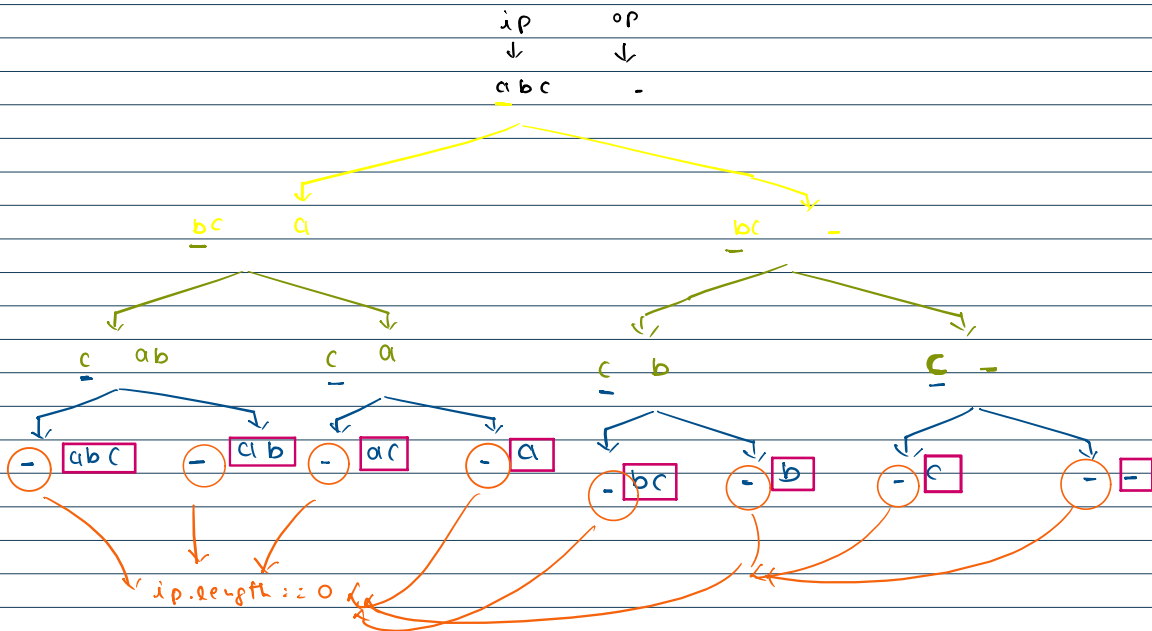
arr → {5, 7, 8, 11, 15}

o/p: → True / False

Print all subsequence

ip → "abc"

op → a, b, c, ab, bc, ac, abc



```
public class PrintAllSubsequences {  
    public static void main(String args[]) {  
        String ip = "abc";  
        String op = "";  
        subsequence(ip, op);  
    }  
}
```

ip = "abc"
op = ""
sub(ip, op)

sub("", "abc")

sub("c", "ab")

abc

```

String ip = "abc";
String op = "";
subsequence(ip, op);
}

3 usages
private static void subsequence(String ip, String op) {
    (ip.length() == 0) {
        System.out.println(op);
        return;
    }
    subsequence(ip.substring(beginIndex: 1), op: op + ip.charAt(0)); // include
    subsequence(ip.substring(beginIndex: 1), op); // don't include
}
}

```

S op(op)	
ip: c op: ab sub(, ab) sub(, ab)	sub(c, ab)
ip: bc op: a sub(c, ab) sub(c, a)	sub(bc, a)
ip: abc op: - sub(bc, a) sub(bc, -)	sub(abc, -)
ip: abc op: - sub(ip, op)	main